

# PROYECTO DE IoT: CONTROL DE INVERNADERO

**Instituto Superior Tecnológico Inmaculada Concepción (ISTIC)**

**Tecnicatura Superior en Análisis de Sistemas**

**Asignatura: Integración de Tecnologías (IN-TE-A)**

**Comisión: A Única**

**Profesor: Carlos F. Gorosito**

**Integrantes: [COMPLETAR NOMBRES Y APELLIDOS]**

**Fecha de entrega: 22 de junio de 2026**

*Antes de entregar: reemplazar el campo de integrantes, revisar los precios con comprobantes locales y agregar fotografías del prototipo físico en el anexo.*

# 1. Introducción

El presente proyecto integra hardware, programación de microcontroladores, comunicación serial, comunicación web en tiempo real y una interfaz de usuario protegida por sesión. El problema elegido es el control de un invernadero a pequeña escala: medir la humedad del suelo, representar su estado y accionar un mecanismo de riego cuando la tierra se encuentra seca.

La solución utiliza un Arduino como controlador físico y una máquina de estados finitos (MEF) con tres estados. Las lecturas viajan por USB hasta una página puente que utiliza WebSerial. Esta página reenvía los eventos a un servidor Node.js con Socket.IO y, finalmente, el dashboard PHP actualiza la medición y el estado sin recargar la página. El flujo inverso permite que una persona autorizada envíe comandos desde el dashboard hacia el Arduino.

El objetivo académico es demostrar una integración completa y comprensible. El sistema no pretende reemplazar un controlador agrícola certificado: los umbrales deben calibrarse y los actuadores de potencia requieren protecciones eléctricas adicionales.

## 2. Desarrollo

### 2.1 Alcance implementado

El repositorio contiene un inicio de sesión PHP, un dashboard web, un puente WebSerial protegido por sesión y un servidor de eventos Socket.IO. También se incorpora como parte de esta entrega un firmware Arduino de referencia compatible con el protocolo utilizado por el software.

El dashboard presenta la lectura del sensor en voltios, una barra de progreso y el estado actual de la MEF. Además, ofrece controles manuales para forzar los estados e0, e1 y e2. El puente de hardware recibe líneas seriales completas mediante el objeto eventosArduino, interpreta los mensajes medida=n y Estado=n, y los retransmite al servidor.

### 2.2 Metodología

Se utilizó una metodología incremental de prototipado:

1. Separación del acceso público y el dashboard mediante sesiones PHP.
2. Implementación del servidor de eventos y prueba del canal Socket.IO.
3. Incorporación de WebSerial para comunicar el navegador con el microcontrolador.
4. Definición de un protocolo de texto simple para lecturas, estados y comandos.
5. Modelado del control físico mediante una MEF de tres estados.
6. Validación sintáctica de PHP y JavaScript y prueba HTTP del inicio de sesión.
7. Preparación de una guía de montaje, calibración y operación.

Esta división facilita localizar fallas: primero se comprueba Serial, luego WebSerial, después Socket.IO y por último la actualización del dashboard.

### 2.3 Destinatarios

La propuesta está dirigida a propietarios de invernaderos domésticos, viveros educativos, huertas escolares y estudiantes que necesiten observar y automatizar un riego básico. El prototipo es apropiado para demostraciones y aprendizaje; para uso productivo se requieren sensores más durables, gabinete, respaldo energético y componentes certificados.

### 2.4 Usuarios y relación con el sistema

El usuario principal es el operador del invernadero. Inicia sesión, consulta la humedad y el estado, abre el puente de hardware y puede forzar un estado durante pruebas. Un segundo perfil conceptual es el técnico, encargado de programar el Arduino, calibrar umbrales, revisar conexiones y mantener los servicios PHP y Node.js.

La sesión PHP evita que una persona sin autenticar abra directamente el puente serial. Las credenciales actuales (abc / 1234) son académicas y están incorporadas en el código; no son adecuadas para producción.

### 3. Hardware requerido

#### 3.1 Componentes

| Elemento                     | Cantidad | Propósito                                             |
|------------------------------|----------|-------------------------------------------------------|
| Arduino Uno o compatible     | 1        | Ejecutar la MEF y gestionar entradas/salidas.         |
| Sensor de humedad de suelo   | 1        | Medir la condición del sustrato por A0.               |
| Módulo LDR                   | 1        | Medir luminosidad por A1 y permitir ampliaciones.     |
| Servomotor SG90              | 1        | Simular o accionar la apertura de una llave de riego. |
| LED rojo y LED verde         | 2        | Señalizar suelo seco o húmedo.                        |
| Buzzer                       | 1        | Avisar el ingreso al estado de tierra seca.           |
| Resistencias de 220 $\Omega$ | 2        | Limitar corriente de los LED.                         |
| Protoboard y jumpers         | 1 juego  | Realizar el prototipo.                                |
| Cable USB                    | 1        | Alimentación y comunicación serial.                   |
| Fuente externa de 5 V        | 1        | Alimentar el servo sin sobrecargar el Arduino.        |

#### 3.2 Conexiones

| Dispositivo       | Pin | Observación                                       |
|-------------------|-----|---------------------------------------------------|
| Sensor de humedad | A0  | VCC a 5 V y GND común.                            |
| Sensor de luz LDR | A1  | VCC a 5 V y GND común.                            |
| Servo             | D9  | Señal en D9; alimentación externa de 5 V.         |
| LED rojo          | D6  | En serie con resistencia de 220 $\Omega$ .        |
| LED verde         | D7  | En serie con resistencia de 220 $\Omega$ .        |
| Buzzer            | D5  | Usar etapa de potencia si el consumo lo requiere. |

Todos los módulos deben compartir referencia GND. Una bomba o electroválvula no debe conectarse directamente al microcontrolador: debe utilizar relé o MOSFET, diodo de rueda libre y fuente dimensionada.

#### 3.3 Precios estimativos

Los siguientes valores son una estimación orientativa en pesos argentinos para elaborar el presupuesto académico. No constituyen una cotización y deben actualizarse con proveedores locales antes de la entrega o compra.

| Elemento | Estimación (ARS) |
|----------|------------------|
|----------|------------------|

|                            |          |
|----------------------------|----------|
| Arduino Uno compatible     | \$20.000 |
| Sensor de humedad          | \$4.000  |
| Módulo LDR                 | \$3.000  |
| Servomotor SG90            | \$8.000  |
| LED, buzzer y resistencias | \$4.000  |
| Protoboard y jumpers       | \$12.000 |
| Cable USB                  | \$5.000  |
| Fuente externa 5 V         | \$9.000  |
| Total orientativo          | \$65.000 |

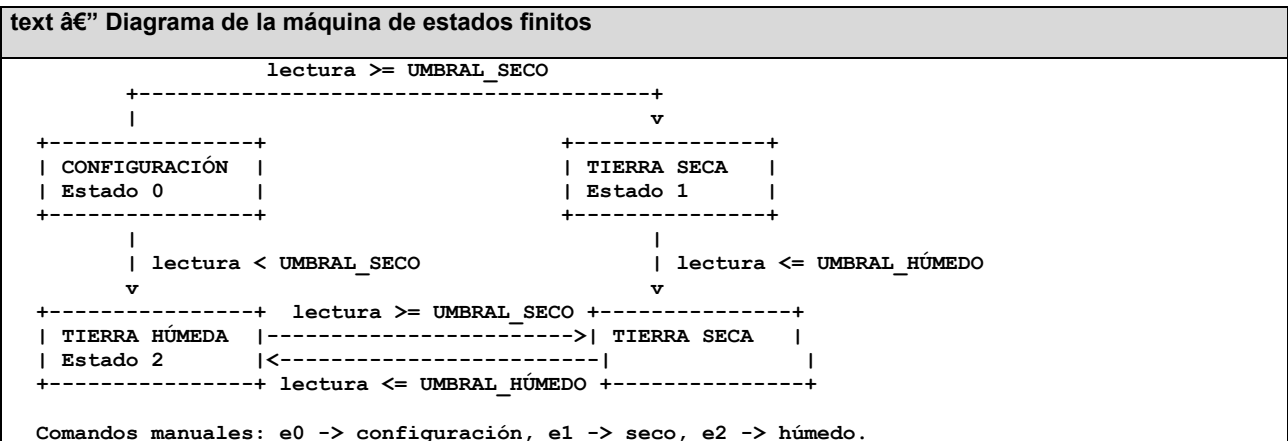
## 4. Funcionamiento del hardware e interacción

Al encenderse, el Arduino ingresa en configuración, cierra el mecanismo de riego y espera dos segundos para estabilizar la lectura. Luego compara el valor de humedad con el umbral seco. Si la tierra está seca, abre el servo, enciende el LED rojo y emite un aviso breve. Si alcanza el umbral húmedo, cierra el servo y enciende el LED verde.

Se utilizan dos umbrales distintos. Esta técnica, llamada histéresis, impide que pequeñas variaciones alrededor de un único límite provoquen aperturas y cierres continuos. Cada segundo se envían la medición cruda y la lectura de luz. Los cambios de estado se informan inmediatamente.

El operador observa los datos en el dashboard. Durante una demostración puede forzar configuración, seco o húmedo. Estos comandos son útiles para probar actuadores, pero no reemplazan el control automático.

## 5. Gráfico y definición de estados



| Estado            | Entrada relevante    | Salidas                               | Transición automática                  |
|-------------------|----------------------|---------------------------------------|----------------------------------------|
| 0 — Configuración | Lectura estabilizada | Servo cerrado, indicadores apagados   | Decide seco/húmedo luego de 2 s.       |
| 1 — Tierra seca   | Humedad cruda alta   | Servo abierto, LED rojo, aviso buzzer | Pasa a húmedo al llegar a 500 o menos. |
| 2 — Tierra húmeda | Humedad cruda baja   | Servo cerrado, LED verde              | Pasa a seco al llegar a 650 o más.     |

## 6. Programa de Arduino

El programa completo se entrega en arduino/control\_invernadero/control\_invernadero.ino. El fragmento siguiente muestra el ciclo principal y conserva la lógica en funciones separadas.

#### Arduino C++ "Ciclo principal de la máquina de estados"

```
void loop() {  
  leerComandos();  
  
  int humedad = analogRead(PIN_HUMEDAD);  
  if (estado == CONFIGURACION) ejecutarConfiguracion(humedad);  
  if (estado == TIERRA_SECA) ejecutarTierraSeca(humedad);  
  if (estado == TIERRA_HUMEDA) ejecutarTierraHumeda(humedad);  
  
  reportarSensores(humedad);  
  delay(100);  
  tiempoEstado += 100;  
}
```

El protocolo serial se mantiene deliberadamente simple:

• medida:512: lectura analógica de humedad entre 0 y 1023.

• Estado=1: estado actual de la MEF.

• luz:700: lectura del LDR, preparada para una versión futura.

• e0, e1, e2: comandos recibidos desde el dashboard.

## 7. Flujo de información enviada al software

#### text "Arquitectura y recorrido bidireccional de los datos"

```
Sensor -> Arduino/MEF -> Serial USB -> WebSerial (PHP)  
        -> evento "arduino-dice" -> Node.js/Socket.IO  
        -> evento "servidor-dice:lectura-sensor-1" -> Dashboard  
  
Dashboard -> "comando-desde-dashboard" -> Socket.IO  
           -> "servidor-envia-accion-placa" -> WebSerial  
           -> Serial USB -> Arduino -> actuadores
```

La medición cruda se convierte a voltios mediante  $\text{valor} / 1023 * 5.00$ . El denominador 1023 corresponde al máximo de una lectura analógica de 10 bits del Arduino Uno. Socket.IO desacopla la pestaña que posee el puerto serial de las pestañas que visualizan el dashboard.

#### JavaScript "Lectura serial y retransmisión por Socket.IO"

```
if (_fraseDelArduino.includes("medida:")) {  
  let valor = +_fraseDelArduino.slice("medida:".length);  
  let voltaje = valor / 1023 * 5.00;  
  _socket.emit("arduino-dice", "sensor-1", voltaje);  
}
```

## 8. Interfaces de software

### 8.1 Inicio de sesión

La pantalla inicial solicita usuario y contraseña. index.php inspecciona la sesión y decide si incluye el formulario o el dashboard. Los errores se guardan temporalmente en la sesión y se muestran durante tres segundos.

#### PHP "index.php — selección de vista según la sesión"

```
session_start();  
$haySesion = isset($_SESSION["usuario"]);  
  
if ($haySesion):  
  include "con-sesion.php";  
else:  
  include "sin-sesion.php";  
endif;
```

## 8.2 Dashboard

Presenta usuario autenticado, voltaje de humedad, barra de progreso, estado de la MEF y botones de prueba. La interfaz se actualiza mediante eventos sin recargar el documento.

## 8.3 Puente de hardware

conectar-hardware.php está protegido por sesión y utiliza el componente <arduino-usb> provisto para la materia. La integración correcta registra un objeto de eventos cuyo método renglon invoca fnLeidoArduino.

| JavaScript â€” Adaptación a la interfaz de eventos del componente arduino-usb                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>window.eventosArduino = {   renglon: fnLeidoArduino,   conectado: function() { /* informar conexión */ },   desconectado: function() { /* informar desconexión */ },   cancelado: function(mensaje) { /* mostrar error */ } };</pre> |

## 9. Herramientas y lenguajes utilizados

| Herramienta o lenguaje | Uso                                               |
|------------------------|---------------------------------------------------|
| Arduino C++            | Lectura de sensores, MEF y control de actuadores. |
| PHP 8.4                | Sesiones, login, logout y composición de vistas.  |
| HTML5 y CSS3           | Estructura y presentación de las interfaces.      |
| JavaScript             | WebSerial, Socket.IO y actualización del DOM.     |
| Node.js                | Ejecución del servidor de tiempo real.            |
| Socket.IO              | Eventos bidireccionales entre puente y dashboard. |
| WebSerial API          | Acceso al puerto serie desde el navegador.        |
| Git y GitHub           | Control de versiones y publicación del proyecto.  |
| Visual Studio Code     | Edición y prueba del código.                      |

## 10. Guía para el usuario

1. Conectar y programar el Arduino según el esquema.
  2. Ejecutar npm install la primera vez.
  3. Iniciar node server.js.
  4. Iniciar php -S 127.0.0.1:8000 en otra terminal.
  5. Abrir http://127.0.0.1:8000/index.php en Chrome o Edge.
  6. Ingresar con abc y 1234.
  7. Abrir el puente, presionar Conectar y seleccionar el puerto.
  8. Regresar al dashboard para observar medidas y estados.
  9. Al finalizar, desconectar el puerto, cerrar sesión y detener ambos servidores.
- La guía ampliada y la solución de problemas están en docs/GUIA\_USUARIO.md.

## 11. Conclusión

El proyecto demuestra que tecnologías con responsabilidades diferentes pueden integrarse mediante un protocolo pequeño y explícito. Arduino mantiene el control local; WebSerial vincula

el hardware con el navegador; Socket.IO distribuye los eventos en tiempo real; PHP restringe el acceso y presenta el dashboard.

La arquitectura permite observar el sistema y actuar sobre él sin acoplar el microcontrolador directamente a la interfaz. También deja claro qué partes requieren trabajo físico: calibración, alimentación del actuador y validación del mecanismo de riego.

## 11.1 Aprendizajes

Se aprendió a diseñar una MEF, interpretar señales analógicas, implementar histéresis, definir mensajes seriales, trabajar con comunicación bidireccional y coordinar sesiones PHP con eventos WebSocket. También se comprobó la importancia de que productor y consumidor compartan exactamente el mismo protocolo: diferencias como Estado=1 frente a estado:1 pueden interrumpir toda la integración.

## 11.2 Futuras versiones

- Persistir mediciones en una base de datos y presentar históricos.
- Incorporar temperatura, humedad ambiente y luminosidad al dashboard.
- Reemplazar credenciales fijas por usuarios con hash y roles.
- Usar HTTPS y restringir CORS.
- Enviar alertas y permitir configuración remota de umbrales.
- Agregar modo manual con tiempo máximo y parada de seguridad.
- Utilizar sensores capacitivos y gabinete resistente a humedad.
- Migrar a ESP32 para comunicación Wi-Fi directa.

## 11.3 Recomendaciones para continuadores

Calibrar antes de automatizar, probar cada capa por separado y registrar el protocolo de mensajes. No alimentar cargas de potencia desde el Arduino. Mantener un modo seguro ante desconexiones y evitar que un comando manual deje el riego abierto indefinidamente. Finalmente, conservar la documentación y el firmware versionados junto con el software.

## 12. Referencias bibliográficas

- Arduino. \*Language Reference: analogRead()\* y documentación de la plataforma. <https://docs.arduino.cc/language-reference/>
- Arduino. \*Servo Library\*. <https://docs.arduino.cc/libraries/servo/>
- MDN Web Docs. \*Web Serial API\*. [https://developer.mozilla.org/docs/Web/API/Web\\_Serial\\_API](https://developer.mozilla.org/docs/Web/API/Web_Serial_API)
- Socket.IO. \*Emitting events\*. <https://socket.io/docs/v4/emit-events/>
- PHP Documentation. \*Sessions\*. <https://www.php.net/manual/en/book.session.php>
- Node.js. \*Documentation\*. <https://nodejs.org/docs/latest/api/>
- Gorosito, Carlos F. (2026). \*Proyecto de IoT — Integración de Tecnologías\*. Material de cátedra, ISTIC.

## 13. Anexo / Apéndice

### 13.1 Repositorio

Código fuente: <https://github.com/arialbitres/Control-Invernadero>

### 13.2 Archivos principales

- index.php: selección de interfaz según sesión.
- sin-sesion.php, login.php, logout.php: autenticación académica.
- con-sesion.php: dashboard en tiempo real.
- conectar-hardware.php: puente WebSerial–Socket.IO.
- server.js: relevo de eventos.

â€¢ arduino/control\_invernadero/control\_invernadero.ino: firmware de referencia.

â€¢ docs/GUIA\_USUARIO.md: operación y solución de problemas.

â€¢ docs/CONEXIONES.md: montaje y calibración.

â€¢ docs/DIAGRAMAS.md: diagramas Mermaid versionables.

### **13.3 Evidencia pendiente de incorporar antes de entregar**

1. Fotografía general del prototipo montado.
2. Fotografía de las conexiones de alimentación y GND común.
3. Captura del monitor serial mostrando medida: y Estado=.
4. Captura del dashboard recibiendo una medición real.
5. Comprobantes o enlaces de precios actualizados.

No se incluyen imágenes ficticias: este apartado debe completarse con evidencia del montaje realizado por el grupo.